

Policy gradient algorithms

CS 5624: Natural Language Processing

Fall 2026

<https://tuvllms.github.io/nlp-fall-2026/>

Tu Vu



Logistics

- Final projects
 - Project proposal
 - Template available on Piazza @23, due on 10/12 (no late days)
 - Potential topics shared on Piazza @24
 - Sample project available on Piazza @21
- HW1 due next Monday 10/5

AI news

Gemini 4 Argon

	Benchmark		Gemini 4 Argon	GPT-6 Astra	Claude Fable 5.1	Claude Opus 5.5
Knowledge work	Vals Index		68.9%	63.1%	65.8%	67.0%
	AutomationBench	Score	51.3%	41.4%	31.4%	42.5%
	Vals Finance Agent v2		65.4%	53.5%	58.9%	58.6%
	Harvey's Legal Agent Benchmark		19.6%	5.4%	6.7%	3.8%
Agentic coding	DeepSWE v1.1		77.9%	74.1%	67.4%	74.2%
	FrontierSWE v2		55.0%	65.5%	56.3%	62.3%
	Vibe Code Bench		91.9%	89.6%	90.3%	90.3%
	Terminal-bench 4.0		57.4%	58.2%	57.9%	66.4%
ML engineering	PostTrainBench		45.3%	44.3%	40.2%	49.3%
Science and math	Terminal-Bench Science 0.1		57.6%	68.1%	52.6%	63.3%
	LABBench 2		88.8%	85.4%	68.6%	73.1%
	RiemannBench		76.0%	72.0%	65.6%	69.6%
Long context	GraphWalks	Up to 128k, BFS (F1)	99.7%	98.7%	91.4%	90.6%
		256k to 1M, BFS (F1)	84.2%	71.8%	65.0%	66.8%
Computer use	Agent's Last Exam	Pass rate	39.5%	34.2%	—	38.2%
	OSWorld-2.0	Offline subset Partial score	69.2%	72.6%	—	—
Multimodal understanding	Chartography		71.6%	71.0%	46.2%	66.3%
	LVBench		91.7%	87.5%	79.7%	83.7%
Cybersecurity	CWE-bench v1		68.0%	68.0%	58.0%	67.0%

Methodology: deepmind.google/models/evals-methodology/gemini-4-argon

Artificial Analysis Intelligence Index ↗

Artificial Analysis Intelligence Index v4.3.2 incorporates 10 evaluations: AA-Briefcase v1.1, GDPval-AA v2.1, AutomationBench-AA, Terminal-Bench 4.0, SciCode, Humanity's Last Exam, GDP.pdf, CritPt, AA-Omniscience, AA-LCR v1.1

↺

🔍

📄

25 of 688 models

↕

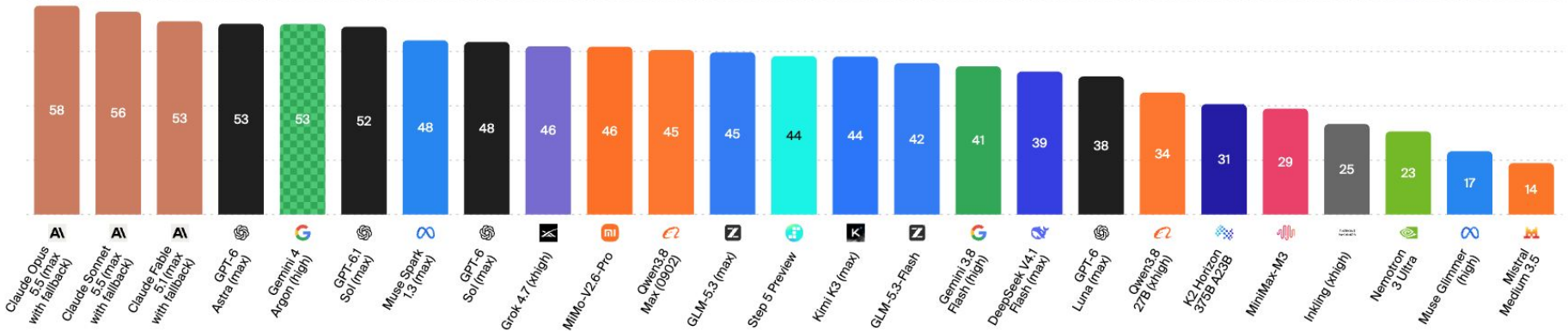
🔍

⌵

+ Add model from specific provider

🔲 Not publicly available

🔲 Artificial Analysis



Context Language Models

Rulin Shao^{1,2}, Shannon Zejiang Shen³, Junjie Oscar Yin^{1,2}, Yuetai Li¹, Minheng Wang¹, Hamish Ivison¹, Radha Poovendran¹, Nathan Lambert⁴, Teng Xiao¹, Mike Lewis², Wen-tau Yih², Luke Zettlemoyer^{1,2}, Pang Wei Koh¹

¹University of Washington, ²Meta Superintelligence Labs, ³MIT, ⁴Trillium Labs

We introduce **Context Language Models (CLMs)**, language models that natively manage their own context. We implement this by treating the **context as a file** and allowing the model to make unrestricted updates to this file. This allows the model to learn what is most important to maintain in context, and naturally extends to multi-agent systems where multiple agent contexts coexist as files. Building CLMs **zero-shot** with existing models outperforms SOTA context-management strategies across a variety of tasks: 11.4% higher accuracy with 21.5% fewer FLOPs on BrowseComp-Plus, 5% higher scores with 59% fewer FLOPs on 12-hour EdgeBench, and 65% greater improvement with the same compute on a 24-hour multi-repository agent-swarm task. Moreover, by shifting context management from external harness control to intrinsic model behavior, CLMs naturally enable both **in-context and parametric learning** of context-management strategies. We show that CLMs can be steered with natural-language instructions evolved through a standard skill-optimization loop, improving held-out accuracy by up to 35.9 points on a context-management task while reducing compute. We also introduce an online reinforcement learning method for CLMs, improving Qwen3.5-9B performance on BrowseComp-Plus by 47.6% while using 12% fewer FLOPs. Finally, we co-design **Suffix Cache Reuse** for CLM serving, further reducing server-side compute by 35% relative to standard SGLang at matched performance.

Date: September 30, 2026

Correspondence: Rulin Shao at rulin@cs.washington.edu

Code: <https://github.com/facebookresearch/context-language-models>

Context Language Models



(A) QUALITATIVE EXAMPLES

Writes loops to prune results

```
for t in turns:
    if irrelevant(t):
        t = "No relevant results."
```

Defines and reuses functions

```
def process_turns(s): ...

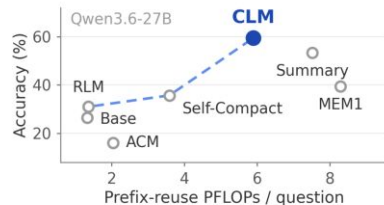
step 109 process_turns(s)
step 142 process_turns(s)
```

Tracks subagents in context

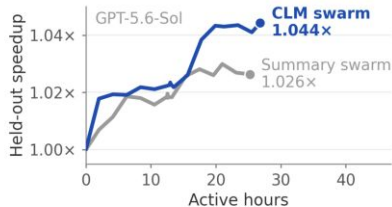
```
## STATE
BEST SO FAR 0.9992
AGENTS 21 · 5 running
```

(B) ZERO-SHOT APPLICATION

BrowseComp-Plus, 32K peak ctx

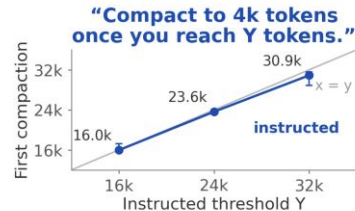


SoftwareWorld, 24hr agent swarm

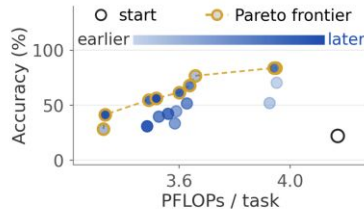


(C) LEARN IN CONTEXT

Steer in context



Evolve in context



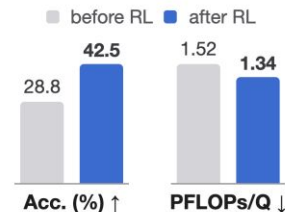
(D) LEARN IN WEIGHTS

Advantage for stepwise GRPO

$$A_i = A_i^{\text{out}} + w_{\text{eff}} \times A_i^{\text{eff}}$$

task advantage success-gated efficiency advantage

BrowseComp-Plus, Qwen3.5-9B



Recursive Language Models

Alex L. Zhang
MIT CSAIL
altzhang@mit.edu

Tim Kraska
MIT CSAIL
kraska@mit.edu

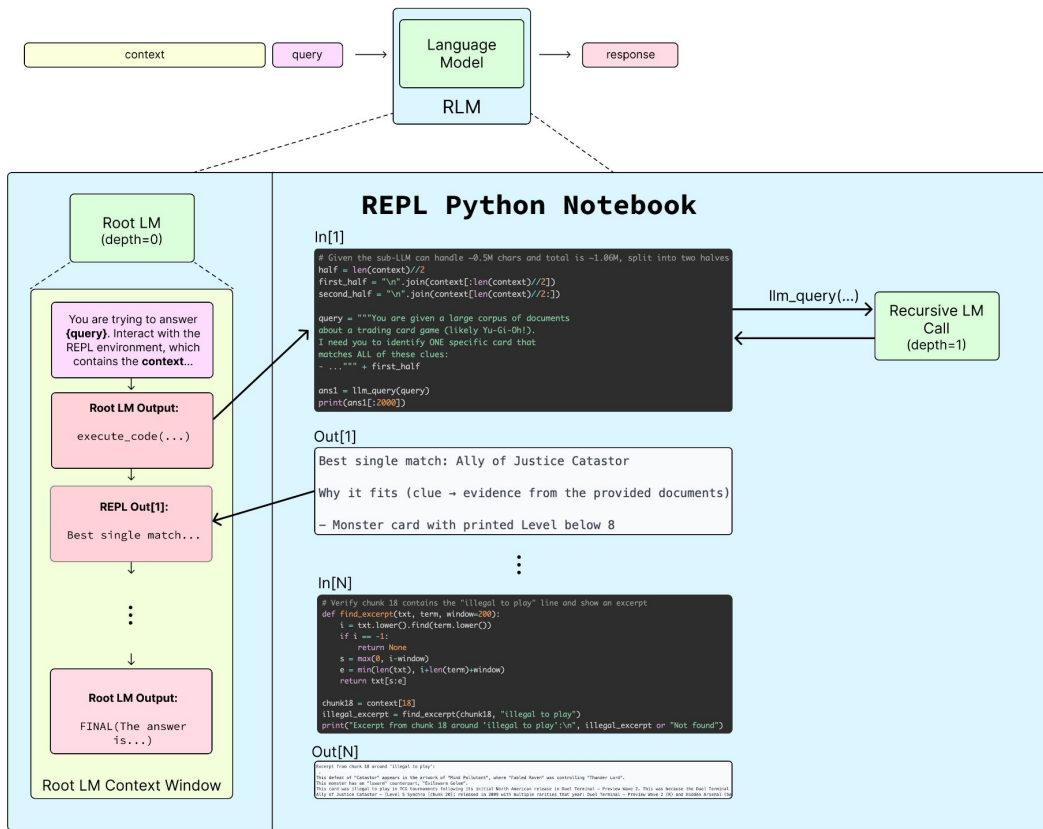
Omar Khattab
MIT CSAIL
okhattab@mit.edu

Abstract

We study allowing large language models (LLMs) to process arbitrarily long prompts through the lens of inference-time scaling. We propose **Recursive Language Models (RLMs)**, a general inference paradigm that treats long prompts as part of an external *environment* and allows the LLM to *programmatically* examine, decompose, and *recursively call itself over* snippets of the prompt. We find that RLMs can successfully process inputs more than an order of magnitude beyond model context window limits and, even for shorter prompts, dramatically outperform the quality of vanilla frontier LLMs and common long-context and coding scaffolds (e.g., on GPT-5 by a median across the evaluated benchmarks of 26% against compaction, 130% against CodeAct with sub-calls, and 13% against Claude Code) across four diverse long-context tasks while having comparable cost. At a small scale, we post-train the first model around the RLM. Our model, **RLM-Qwen3-8B**, outperforms the underlying Qwen3-8B model by a median of 28% and even approaches the quality of vanilla GPT-5 on three long-context tasks. Code is available at <https://github.com/alexzhang13/r1m>.

Recursive Language Models

RLM with a REPL environment:



<https://alexzhang13.github.io/blog/2025/rlm/>

Recursive Language Models (cont'd)

Input: a long report of 10,000 words about a company's 5-year strategy

Query: What risks did the report identify in year 3?

AI: **High-level scan / peek** at the report's table of contents

```
# find the header line number
hdr=$(grep -inE '^(table of contents|contents|toc)\s*$' doc.txt | head -1 | cut -d: -f1)
# print the next 100 lines that look like TOC entries
tail -n +"$hdr" doc.txt | head -n 100 | grep -nE '(\s*\d+(\.\d+)*\s+.)|(\s*.\s+\.{2,}\s*\d+\s*)'
```

Select sub-chunks

Based on that peek, it might decide: “Chapter 4 (pages 40–60) and Chapter 6 (pages 90–110) are likely to mention Year 3 risks.”

Recursive call on chunks It then issues sub-queries:

“From pages 40–60, what risks are mentioned for year 3?”

“From pages 90–110, what risks are mentioned for year 3?”

Aggregate / refine

“In year 3 the report warned of supply chain disruption, regulatory changes, and declining demand in Asia.”

Recursive Language Models (cont'd)

Input: a long report of 10,000 words about a company's 5-year strategy

Query: What risks did the report identify in year 3?

AI: **High-level scan / peek** at the report's table of contents

```
# find the header line number
hdr=$(grep -inE '^(table of contents|contents|toc)\s*$' doc.txt | head -1 | cut -d: -f1)
# print the next 100 lines that look like TOC entries
tail -n +"$hdr" doc.txt | head -n 100 | grep -nE '(\s*\d+(\.\d+)*\s+.+)|(\s*.\s+\.{2,}\s*\d+\s*)'
```

Select sub-chunks

Based on that peek, it might decide: “Chapter 4 (pages 40–60) and Chapter 6 (pages 90–110) are likely to mention Year 3 risks.”

Recursive call on chunks It then issues sub-queries:

“From pages 40–60, what risks are mentioned for year 3?”

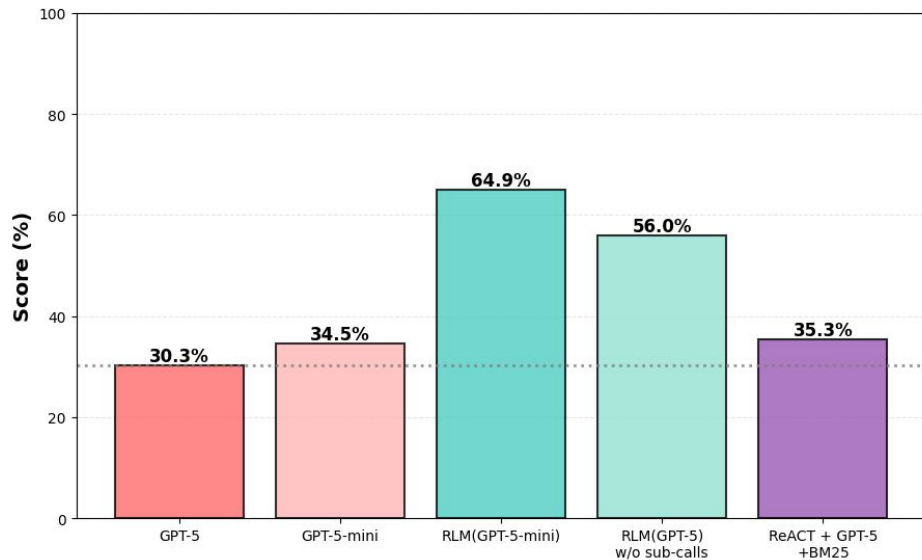
“From pages 90–110, what risks are mentioned for year 3?”

Aggregate / refine

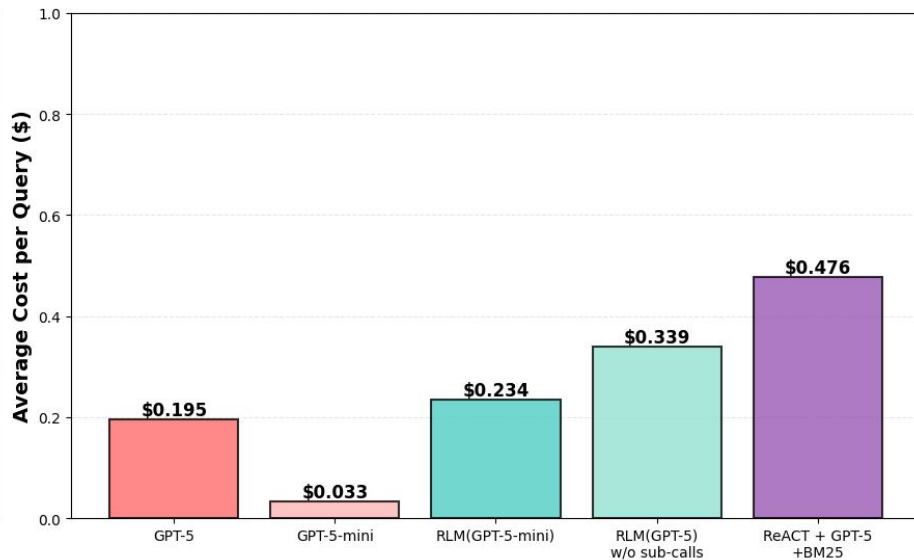
“In year 3 the report warned of supply chain disruption, regulatory changes, and declining demand in Asia.”

Recursive Language Models (cont'd)

OOLONG trec_coarse w/ 132k Token Context - Score (%)

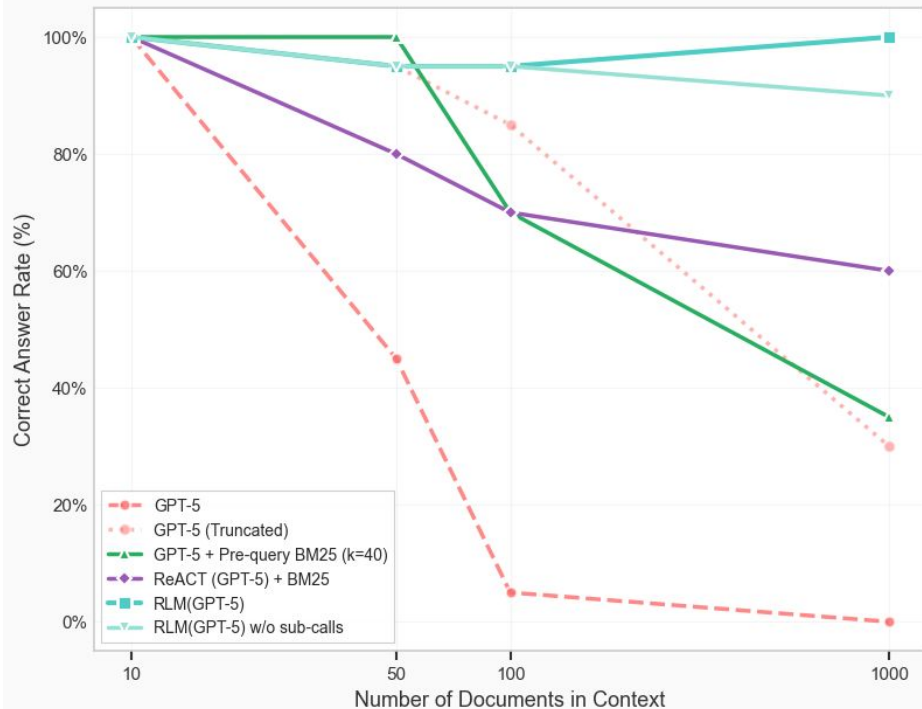


OOLONG trec_coarse w/ 132k Token Context - Avg. Cost (\$)

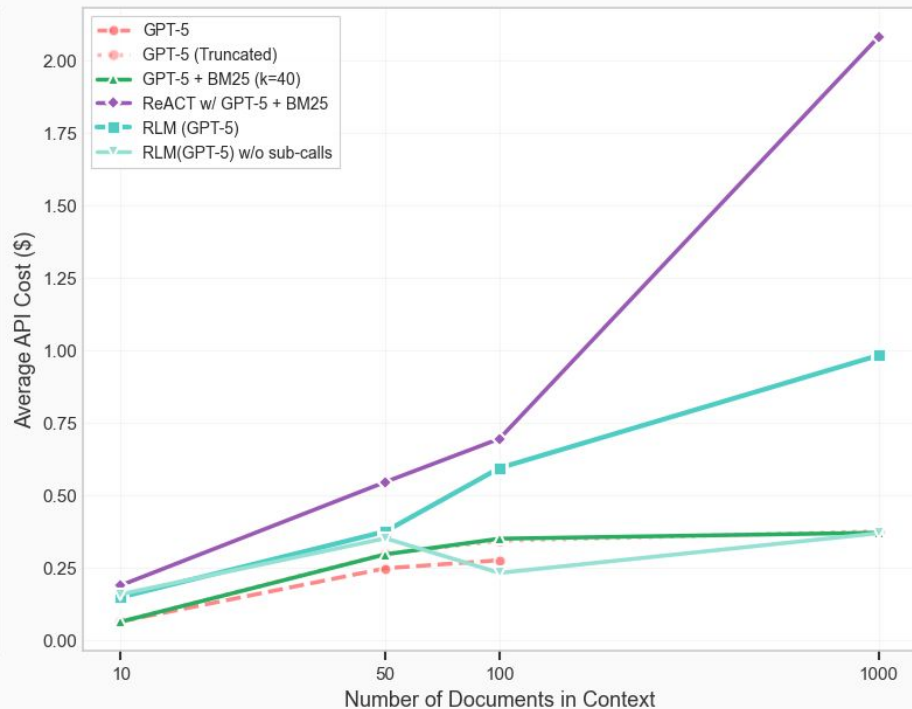


Recursive Language Models (cont'd)

Score on BrowseComp-Plus vs # Context Docs



Average API Cost(\$) per Query vs # Context Docs



Context Language Models



(A) QUALITATIVE EXAMPLES

Writes loops to prune results

```
for t in turns:
    if irrelevant(t):
        t = "No relevant results."
```

Defines and reuses functions

```
def process_turns(s): ...

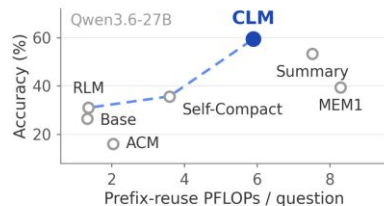
step 109 process_turns(s)
step 142 process_turns(s)
```

Tracks subagents in context

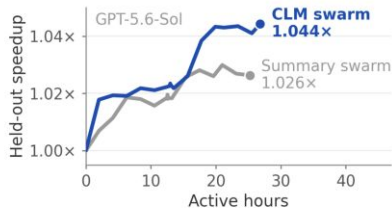
```
## STATE
BEST SO FAR 0.9992
AGENTS 21 · 5 running
```

(B) ZERO-SHOT APPLICATION

BrowseComp-Plus, 32K peak ctx



SoftwareWorld, 24hr agent swarm

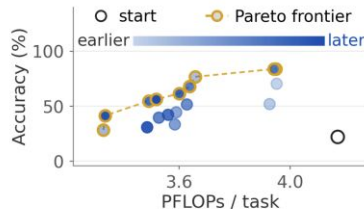


(C) LEARN IN CONTEXT

Steer in context



Evolve in context



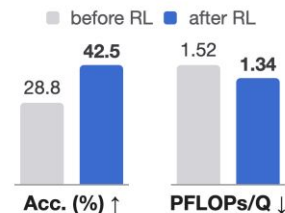
(D) LEARN IN WEIGHTS

Advantage for stepwise GRPO

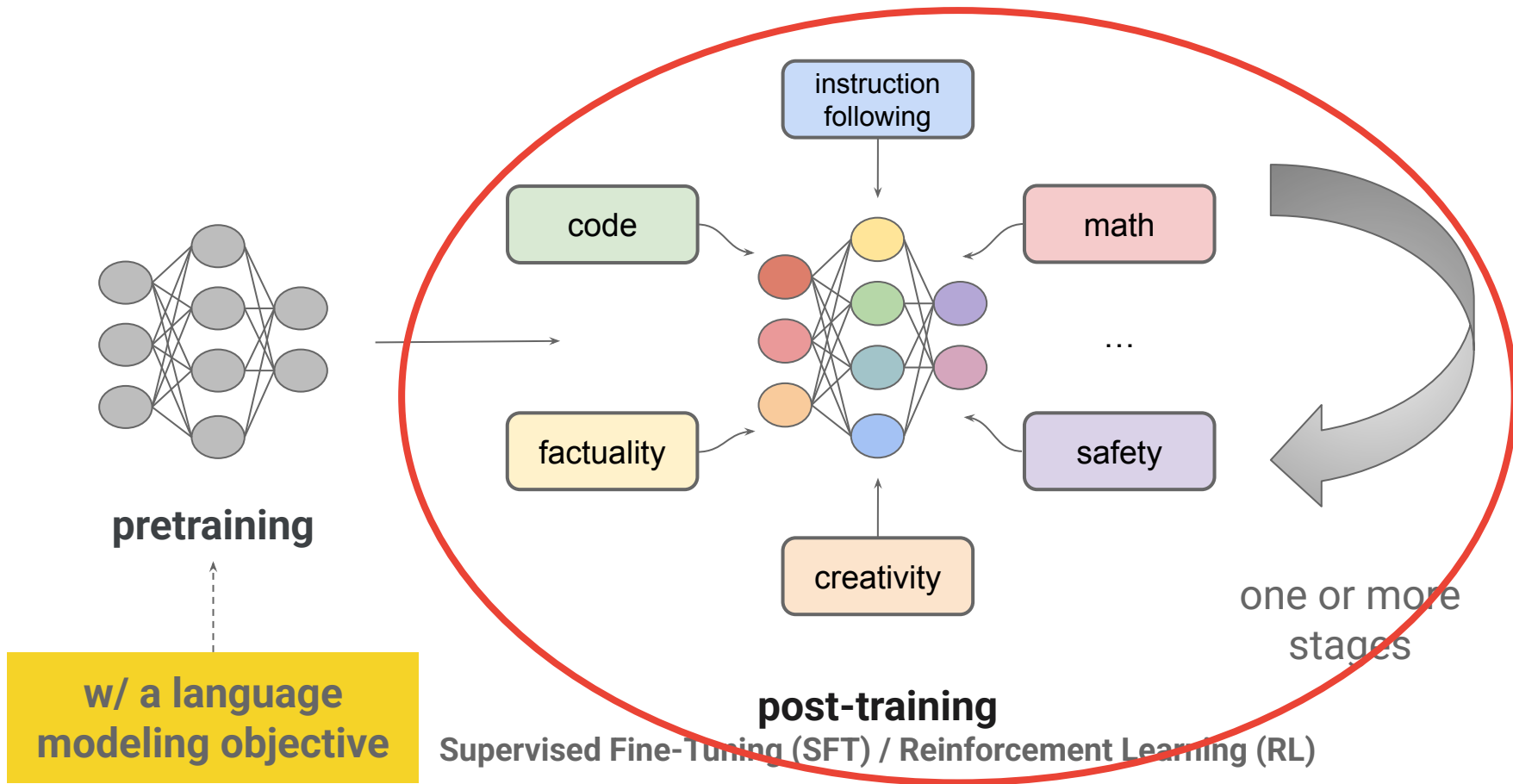
$$A_i = A_i^{\text{out}} + w_{\text{eff}} \times A_i^{\text{eff}}$$

task advantage success-gated efficiency advantage

BrowseComp-Plus, Qwen3.5-9B



Where are we?



Reinforcement learning from human feedback (RLHF)

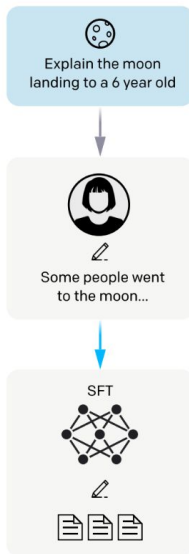
Step 1

Collect demonstration data, and train a supervised policy.

A prompt is sampled from our prompt dataset.

A labeler demonstrates the desired output behavior.

This data is used to fine-tune GPT-3 with supervised learning.



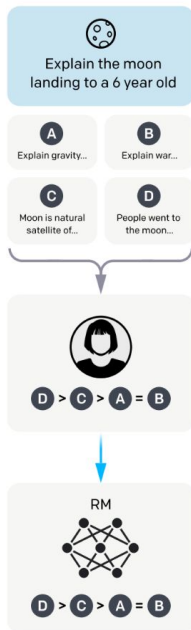
Step 2

Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.

A labeler ranks the outputs from best to worst.

This data is used to train our reward model.



Step 3

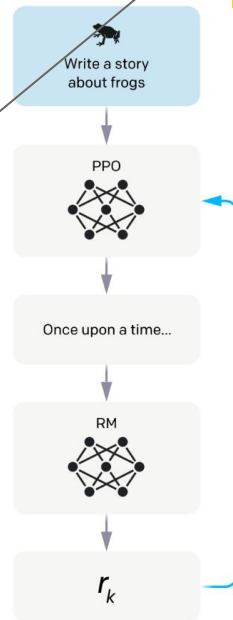
Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.

The policy generates an output.

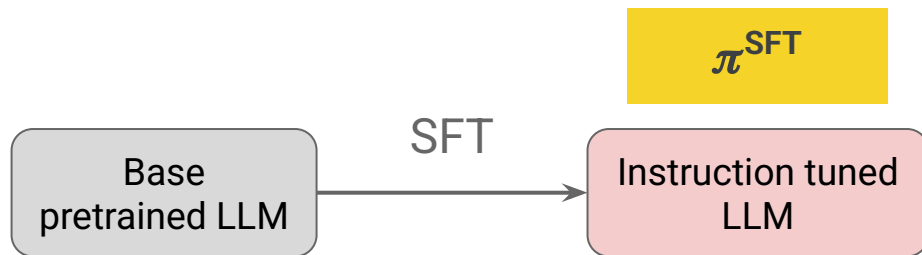
The reward model calculates a reward for the output.

The reward is used to update the policy using PPO.



Learn from data generated by the current policy

Step 1: SFT



Step 2: Reward modelling

Step 2

**Collect comparison data,
and train a reward model.**

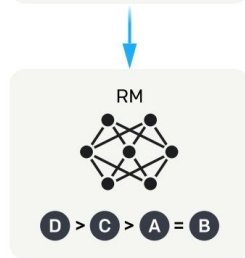
A prompt and
several model
outputs are
sampled.



A labeler ranks
the outputs from
best to worst.



This data is used
to train our
reward model.



Step 2: The Bradley-Terry model

The preferences are assumed to be generated by some latent reward model $r^*(y, x)$, which we do not have access to.

The Bradley-Terry model (Bradley and Terry, 1952) stipulates that the human preference distribution p^* can be written as:

$$p^*(y_1 \succ y_2 | x) = \frac{\exp(r^*(x, y_1))}{\exp(r^*(x, y_1)) + \exp(r^*(x, y_2))}$$

Step 2: Maximum likelihood

Assuming access to a static dataset of comparisons $D = \{x^{(i)}, y_w^{(i)}, y_l^{(i)}\}_{i=1}^N$ sampled from p^* , we can parametrize a reward model $r_\phi(x, y)$ and estimate the parameters via maximum likelihood.

Framing the problem as a binary classification, we have the negative log-likelihood loss:

$$L_R(r_\phi, D) = -\mathbb{E}_{(x, y_w, y_l) \sim D} [\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))]$$

where σ is the logistic function.

$r_\phi(x, y)$ is often initialized from the SFT model $\pi^{\text{SFT}}(y \mid x)$ with an added linear layer on top of the final transformer layer to output a single scalar reward prediction.

Step 3

Step 3

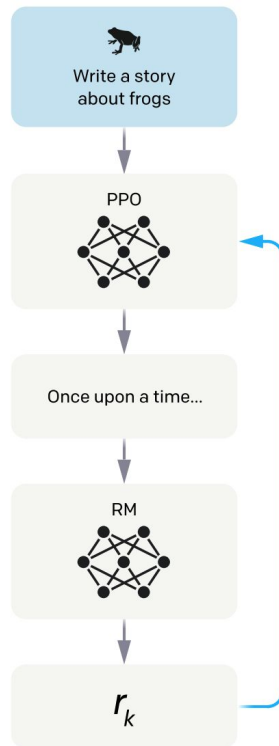
Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.

The policy generates an output.

The reward model calculates a reward for the output.

The reward is used to update the policy using PPO.



Step 3: RL training

The second term prevents the model from deviating too far from the distribution on which the reward model is accurate.

$y \sim \pi_\theta(y|x)$
rollout

$$\max_{\pi_\theta} \mathbb{E}_{x \sim D, y \sim \pi_\theta(y|x)} [r_\phi(x, y)] - \beta D_{KL} [\pi_\theta(y|x) || \pi_{\text{ref}}(y|x)]$$

where β is a parameter controlling the deviation from the base reference policy π_{ref} , namely the initial SFT model π^{SFT} . In practice, the language model policy π_θ is also initialized to π^{SFT} .

Expectation

The **expectation** (or expected value) of a random variable X is

$$\mathbb{E}[X] = \sum_x x P(X = x)$$

where:

- x is a possible value of X
- $P(X = x)$ is the probability of that value
- $\sum_x$ means "sum over all possible values of X "

So expectation is simply a **probability-weighted average**.

Example: fair six-sided die

Let X be the number rolled. Each outcome has probability $1/6$:

$$\begin{aligned}\mathbb{E}[X] &= 1 \left(\frac{1}{6}\right) + 2 \left(\frac{1}{6}\right) + \cdots + 6 \left(\frac{1}{6}\right) \\ &= \frac{1 + 2 + 3 + 4 + 5 + 6}{6} \\ &= \boxed{3.5}.\end{aligned}$$

This does **not** mean you can roll a 3.5. It means that the average roll approaches 3.5 over many rolls.

The **expectation** of a function $f(y)$ under a probability distribution $P(y)$ is defined as:

$$\mathbb{E}_{y \sim P(y)}[f(y)] = \sum_y P(y) f(y)$$

Assume two different distributions for predicting the next word:

- P (from Model 1):
 - $mat \rightarrow 0.7$
 - $floor \rightarrow 0.2$
 - $chair \rightarrow 0.1$
- Q (from Model 2):
 - $mat \rightarrow 0.5$
 - $floor \rightarrow 0.3$
 - $chair \rightarrow 0.2$

Kullback–Leibler (KL) Divergence Calculation

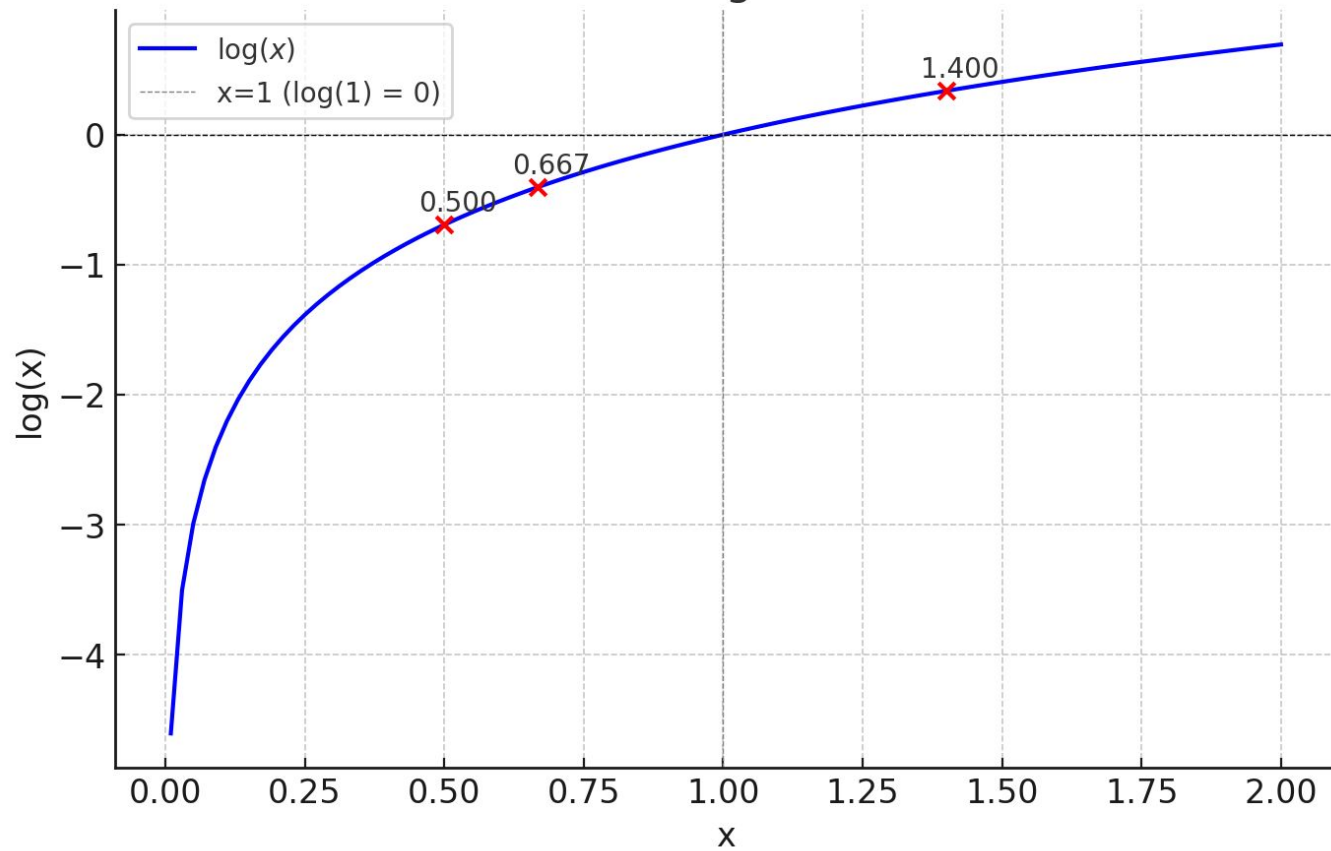
KL divergence measures how much P diverges from Q :

$$D_{KL}(P||Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$$

Substituting the values:

$$D_{KL}(P||Q) = 0.7 \log \frac{0.7}{0.5} + 0.2 \log \frac{0.2}{0.3} + 0.1 \log \frac{0.1}{0.2}$$

Natural Log Function



Cross-entropy loss (cont'd)

The predicted probabilities

$$y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_V \end{bmatrix}$$

The ground truth label

$$\hat{y} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_V \end{bmatrix}$$

$$y_i = \begin{cases} 1, & \text{if } i = c \text{ (correct class index)} \\ 0, & \text{otherwise} \end{cases}$$

$$\hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^V e^{z_j}}, \quad \text{for } i = 1, 2, \dots, V$$

Cross-entropy loss (cont'd)

$$L_{CE}(\hat{y}, y) = - \sum_{i=1}^V y_i \log \hat{y}_i$$

$$L_{CE}(\hat{y}, y) = - (y_1 \log \hat{y}_1 + y_2 \log \hat{y}_2 + \cdots + y_V \log \hat{y}_V)$$

Since the true label y is one-hot encoded, only one term in the sum is nonzero, corresponding to the correct class c , where $y_c = 1$ and $y_i = 0$ for all $i \neq c$. This simplifies the sum to:

$$L_{CE}(\hat{y}, y) = -y_c \log \hat{y}_c$$

Since $y_c = 1$, this further reduces to:

$$L_{CE}(\hat{y}, y) = -\log \hat{y}_c$$

The loss models the distance between the system output and the gold output—lower is better

Let $p(x)$ denote the true distribution and $q(x)$ denote a model distribution.

Cross entropy

$$H(p, q) = - \sum_x p(x) \log q(x)$$

Entropy

$$H(p) = - \sum_x p(x) \log p(x)$$

KL divergence

$$D_{KL}(p||q) = \sum_x p(x) \log \frac{p(x)}{q(x)}$$

These quantities satisfy the identity:

$$H(p, q) = H(p) + D_{KL}(p||q)$$

This decomposition shows why KL divergence is relevant.

- The entropy $H(p)$ depends only on the true distribution.
- When optimizing a model, $H(p)$ is constant.
- Therefore, minimizing cross entropy with respect to q is exactly equivalent to minimizing KL divergence $D_{KL}(p||q)$.

In supervised learning, the empirical distribution plays the role of p . Cross entropy loss, which is used for classification, therefore minimizes the KL divergence between the empirical data distribution and the model distribution.

The entropy measures how uncertain, unpredictable, or spread out a distribution is.

Intuition

Entropy is the expected amount of information in a draw from the distribution.

- If one outcome has probability 1, then there is no uncertainty. The entropy is 0.
- If all outcomes are equally likely, uncertainty is maximal. The entropy is large.
- If probability mass is concentrated on a few outcomes, entropy is lower.

In other words, entropy quantifies how surprised you are on average when you observe a sample.

Cross entropy differs from KL divergence only by a constant that does not depend on the model. That is why minimizing cross entropy is the same objective as minimizing KL divergence.

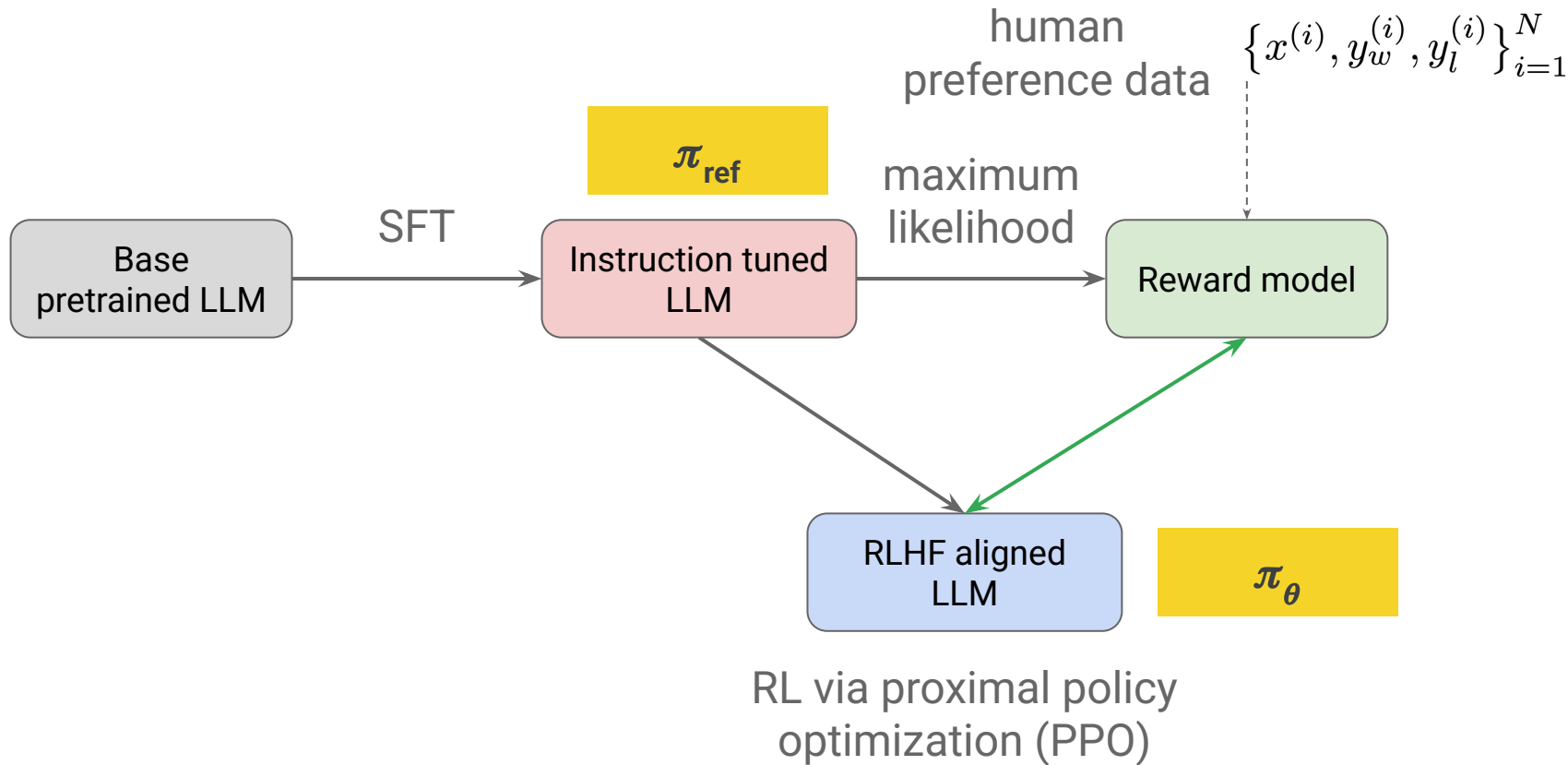
Step 3: RL fine-tuning (cont'd)

The sequence reward is distributed across tokens. PPO (Proximal Policy Optimization) updates happen at the token level.

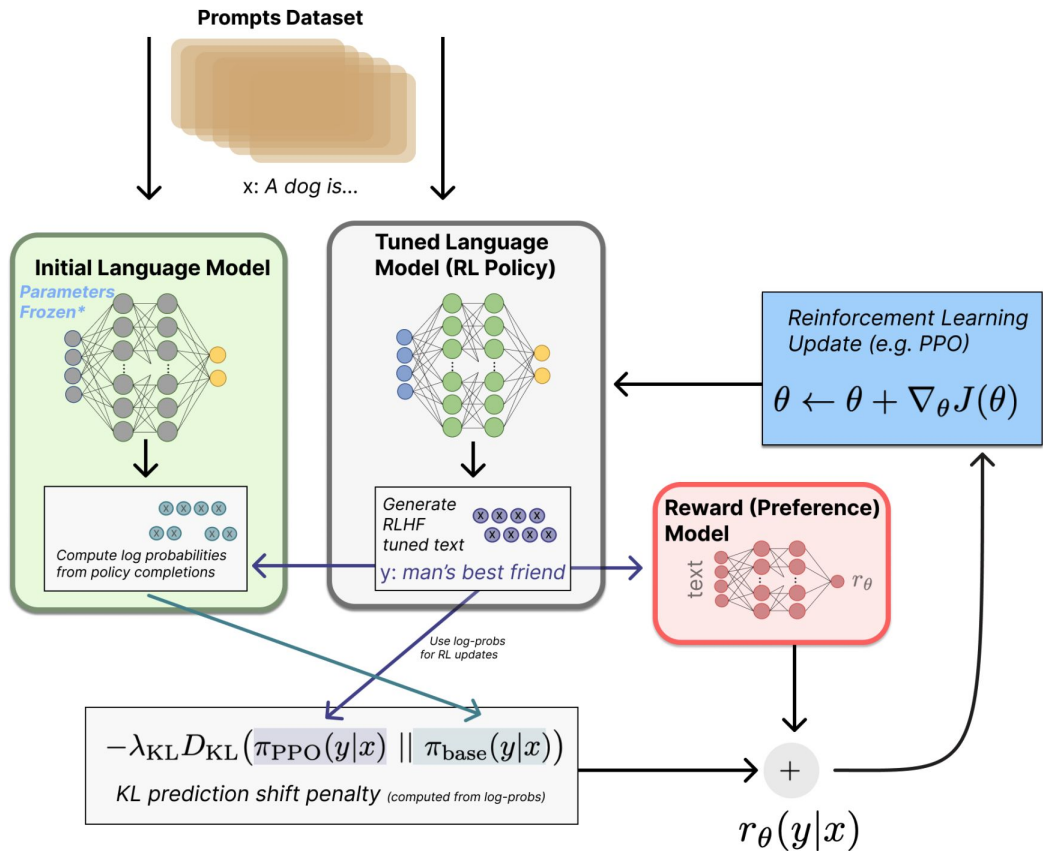

$$\max_{\pi_{\theta}} \mathbb{E}_{x \sim D, y \sim \pi_{\theta}(y|x)} [r_{\phi}(x, y)] - \beta D_{KL} [\pi_{\theta}(y|x) || \pi_{\text{ref}}(y|x)]$$

where β is a parameter controlling the deviation from the base reference policy π_{ref} , namely the initial SFT model π^{SFT} . In practice, the language model policy π_{θ} is also initialized to π^{SFT} .

RLHF pipeline: putting it all together



RLHF pipeline: putting it all together



The effects of RLHF on LLM generalization & diversity

SFT Memorizes, RL Generalizes: A Comparative Study of Foundation Model Post-training

Tianzhe Chu^{♠*} Yuexiang Zhai^{♥♠*} Jihan Yang[♦] Shengbang Tong[♦]
Saining Xie^{♠♦} Dale Schuurmans^{♠✧} Quoc V. Le[♠] Sergey Levine[♥] Yi Ma^{♠♥}

Abstract

Supervised fine-tuning (SFT) and reinforcement learning (RL) are widely used post-training techniques for foundation models. However, their respective role in enhancing model generalization in rule-based reasoning tasks remains unclear. This paper studies the comparative effect of SFT and RL on generalization and memorization, focusing on text-based and visual reason-

1. Introduction

Although SFT and RL are both widely used for foundation model training (OpenAI, 2023b; Google, 2023; Jaech et al., 2024; DeepSeekAI et al., 2025), their distinct effects on *generalization* (Bousquet & Elisseff, 2000; Zhang et al., 2021) remain unclear, making it challenging to build reliable and robust AI systems. A key challenge in analyzing the generalizability of foundation models (Bommasani et al., 2021; Brown et al., 2020) is to separate data mem-

UNDERSTANDING THE EFFECTS OF RLHF ON LLM GENERALISATION AND DIVERSITY

Robert Kirk^{* α} Ishita Mediratta ^{β} Christoforos Nalmpantis ^{β} Jelena Luketina ^{γ}

Eric Hambro ^{β} Edward Grefenstette ^{α} Roberta Raileanu ^{β}

^{α} University College London, ^{β} Meta, ^{γ} University of Oxford

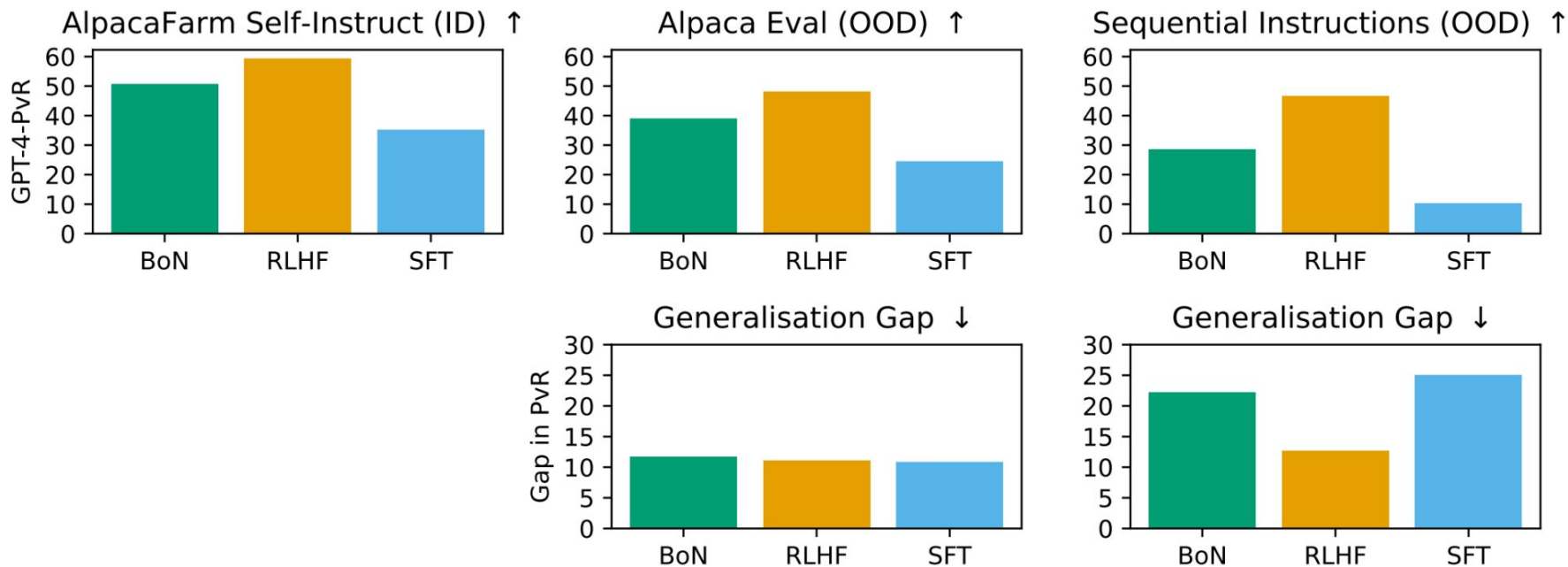


Figure 3: **Instruction Following Generalisation Results.** GPT-4 PvR for SFT, BoN and RL policies, based on LLaMa 7B, trained on the AlpacaFarm Self-Instruct instruction following task. ID is on AlpacaFarm Self-Instruct, OOD is on the AlpacaEval and Sequential Instructions datasets respectively, and generalisation gap is ID – OOD performance.

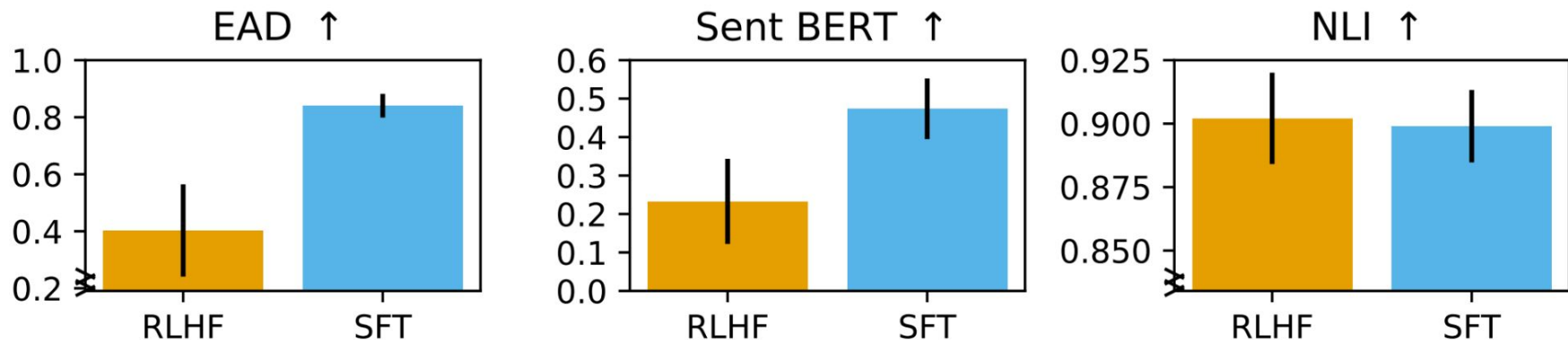


Figure 5: **Per-input diversity metrics for RLHF and SFT models.** For these scores the outputs used to calculate the diversity are a sample of outputs from the model for single input. These per-input scores are then averaged, as in Eq. (2). Error bars are standard deviation of the per-input diversity score across different inputs. Note that some plots have broken y-axis for better visualisation.



Sasha Rush ✓

@srush_nlp

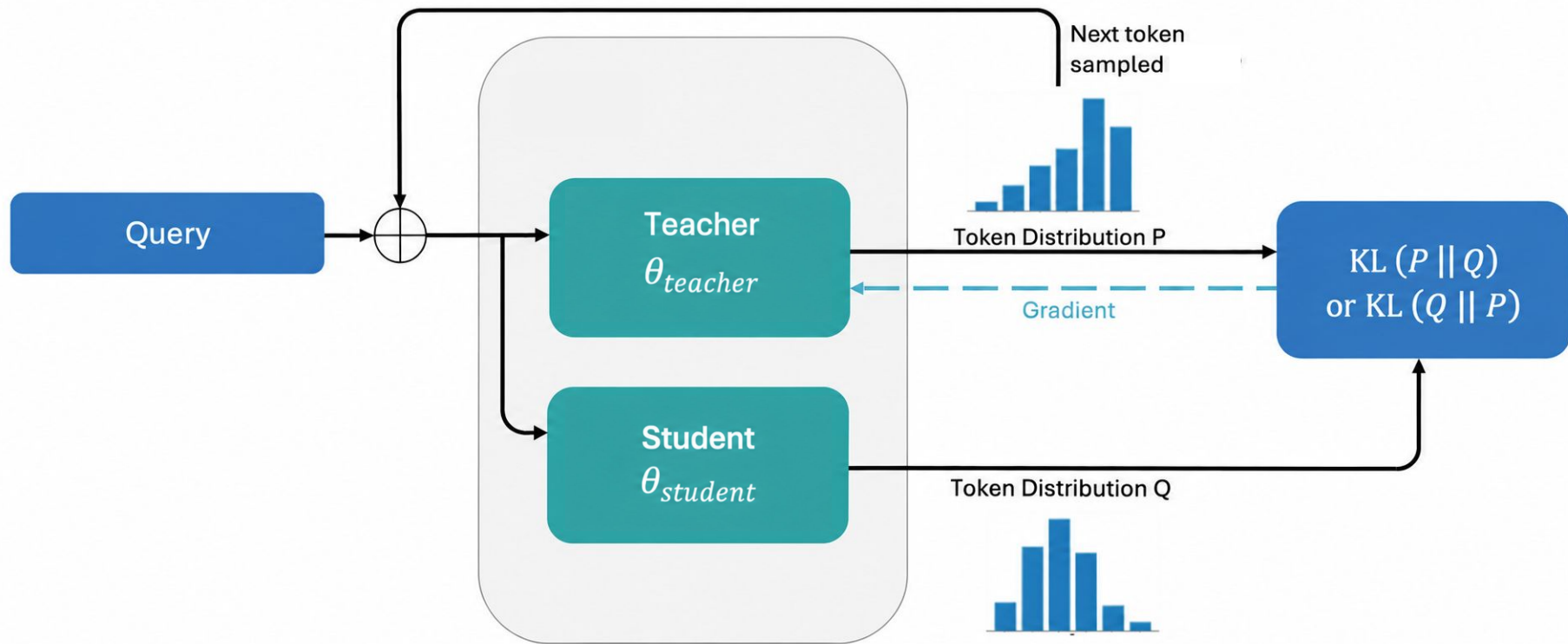


There is significant discussion in the academic literature about RL making models better at pass@1 and *worse* at pass@N (or related claims).

We run a lot of RL runs at Cursor and don't see this issue systematically. Not doubting it occurs, but something else might be going on.

7:02 PM · Dec 18, 2025 · **130K** Views

Knowledge distillation



Let us call:

- $p(x)$ = teacher distribution
- $q(x)$ = student distribution

There are two common ways to match them:

- **Forward KL:** $\text{KL}(p||q)$
- **Reverse KL:** $\text{KL}(q||p)$

They behave very differently.

The **expectation** of a function $f(y)$ under a probability distribution $P(y)$ is defined as:

$$\mathbb{E}_{y \sim P(y)}[f(y)] = \sum_y P(y) f(y)$$

1. Forward KL: mode covering

$$\text{KL}(p||q) = \mathbb{E}_{x \sim p} \left[\log \frac{p(x)}{q(x)} \right]$$

You sample from the **teacher**, then ask:

Can you imitate the teacher everywhere?

2. Reverse KL: mode seeking

$$\text{KL}(q||p) = \mathbb{E}_{x \sim q} \left[\log \frac{q(x)}{p(x)} \right]$$

You sample from the **student**, then ask:

Where you already speak, can you adjust slightly?

The student sticks to its own peaks instead of spreading probability across all the teacher's peaks

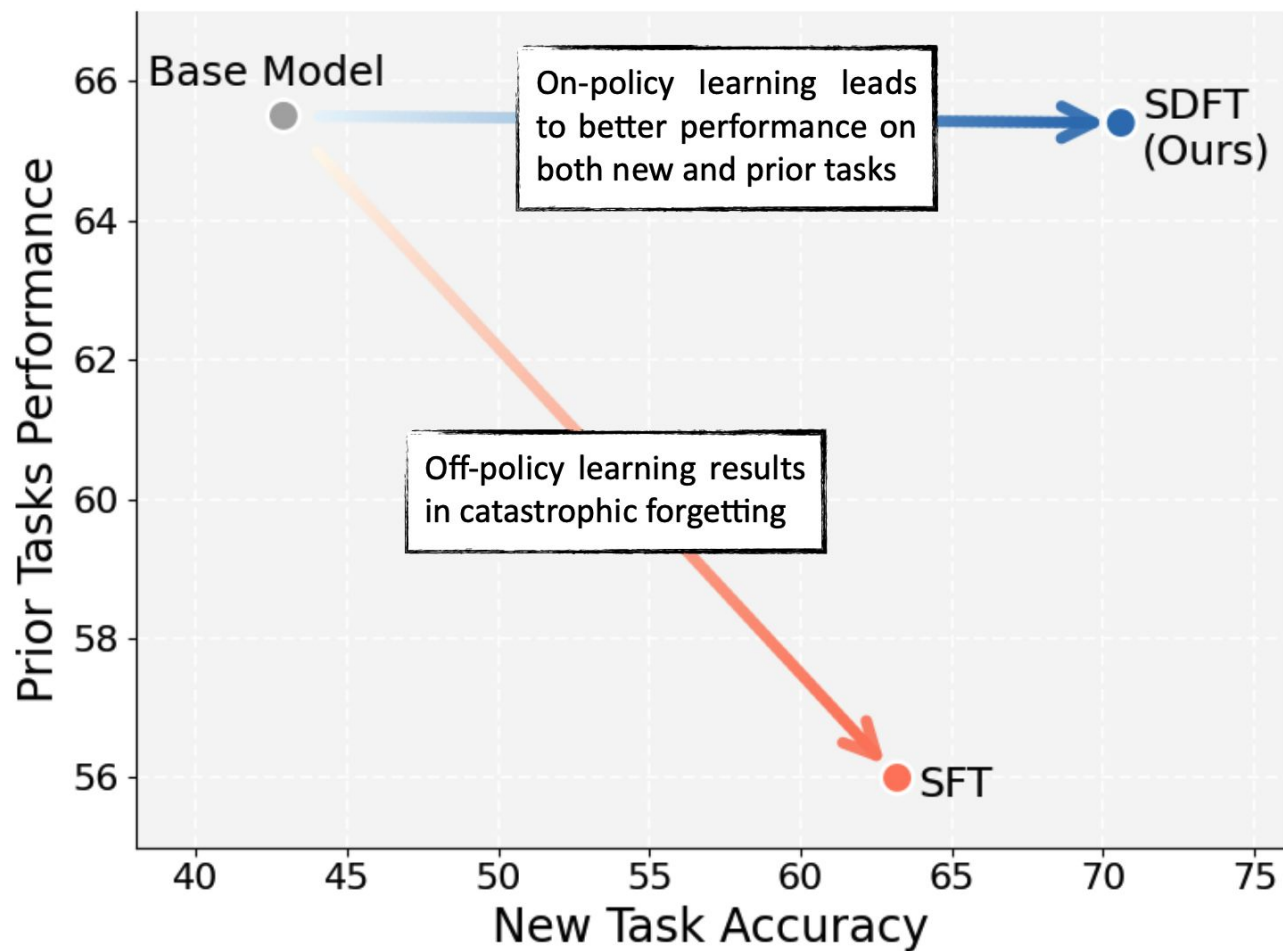
SELF-DISTILLATION ENABLES CONTINUAL LEARNING

Idan Shenfeld^{1 2*} **Mehul Damani**¹ **Jonas Hübötter**³ **Pulkit Agrawal**^{1 2}

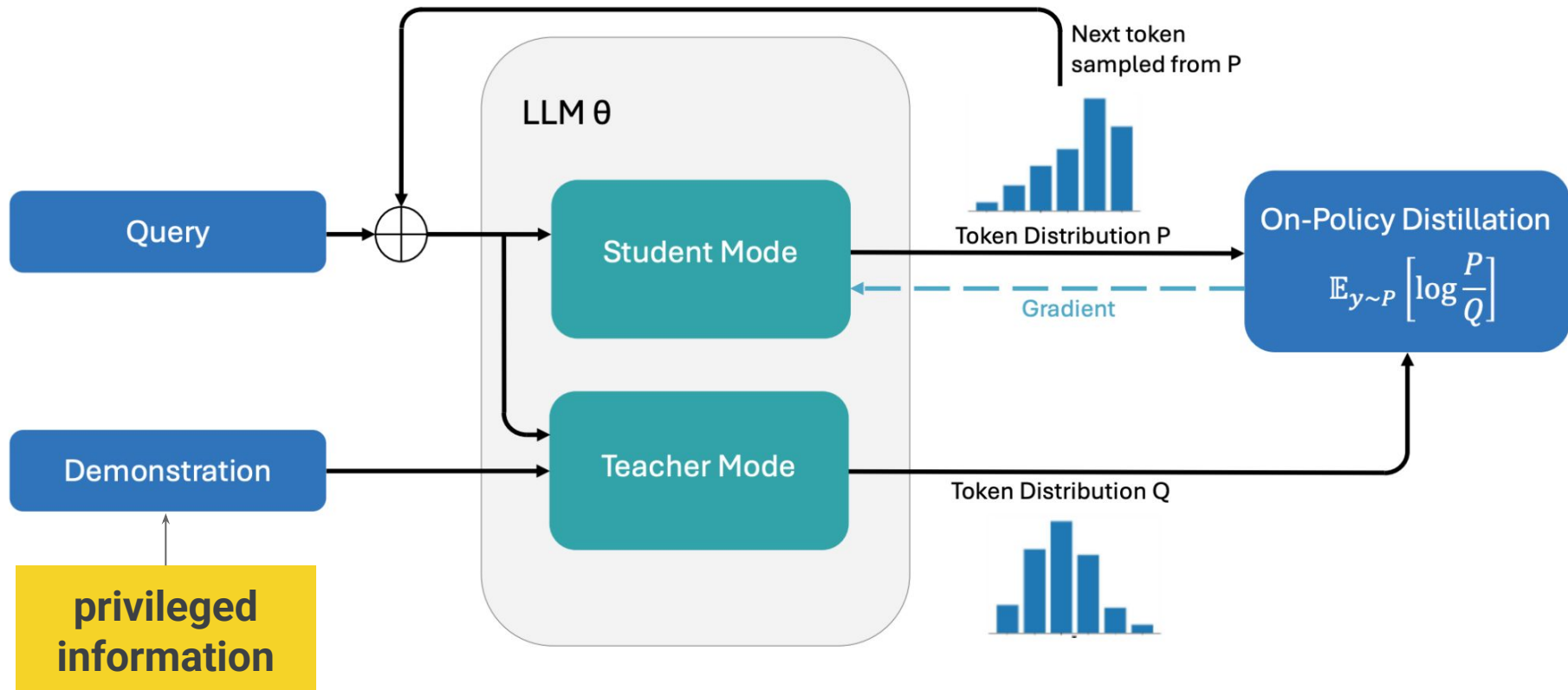
¹MIT ²Improbable AI Lab ³ETH Zurich

ABSTRACT

Continual learning, enabling models to acquire new skills and knowledge without degrading existing capabilities, remains a fundamental challenge for foundation models. While on-policy reinforcement learning can reduce forgetting, it requires explicit reward functions that are often unavailable. Learning from expert demonstrations, the primary alternative, is dominated by supervised fine-tuning (SFT), which is inherently off-policy. We introduce **Self-Distillation Fine-Tuning (SDFT)**, a simple method that enables on-policy learning directly from demonstrations. SDFT leverages in-context learning by using a demonstration-conditioned model as its own teacher, generating on-policy training signals that preserve prior capabilities while acquiring new skills. Across skill learning and knowledge acquisition tasks, SDFT consistently outperforms SFT, achieving higher new-task accuracy while substantially reducing catastrophic forgetting. In sequential learning experiments, SDFT enables a single model to accumulate multiple skills over time without performance regression, establishing on-policy distillation as a practical path to continual learning from demonstrations. Code and Datasets are available at <http://idanshenfeld.com/SDFT>.



Self-Distillation Fine-Tuning



Reinforcement Learning

Policy Gradient Algorithms

Goal: we want a language model (the policy) that produces completions that get high reward.

- Policy $\pi_{\theta}(y|x)$: a distribution over completions given a prompt.
- Reward $R(x, y)$: a scalar that scores the full completion.
- We optimize expected reward:

$$J(\theta) = \mathbb{E}_{y \sim \pi_{\theta}(\cdot|x)}[R(x, y)].$$

Key framing for LMs: the completion is often treated as one action (bandit view), even though we write

$$J(\theta) = \sum_y \pi_{\theta}(y|x) R(x, y).$$

Policy Gradient Algorithms

Goal: we want a language model (the policy) that produces completions that get high reward.

- Policy $\pi_{\theta}(y|x)$: a distribution over completions given a prompt.
- Reward $R(x, y)$: a scalar that scores the full completion.
- We optimize expected reward:

$$J(\theta) = \mathbb{E}_{y \sim \pi_{\theta}(\cdot|x)}[R(x, y)].$$

$$J(\theta) = \sum_y \pi_{\theta}(y|x) R(x, y).$$

Policy Gradient

$$J(\theta) = \sum_y \pi_\theta(y|x) R(x, y).$$

$$d(\log x) = \frac{1}{x} dx$$

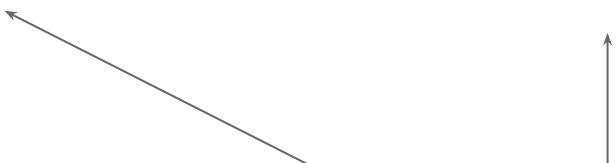
$$\nabla_\theta J(\theta) = \sum_y \nabla_\theta \pi_\theta(y|x) R(x, y).$$

$$\nabla_\theta \pi_\theta(y|x) = \pi_\theta(y|x) \nabla_\theta \log \pi_\theta(y|x).$$

$$dx = x d(\log x)$$

$$\nabla_\theta J(\theta) = \sum_y \pi_\theta(y|x) \nabla_\theta \log \pi_\theta(y|x) R(x, y).$$

Log-derivative trick



$$\boxed{\nabla_\theta J(\theta) = \mathbb{E}_{y \sim \pi_\theta} [\nabla_\theta \log \pi_\theta(y|x) R(x, y)]}$$

Policy Gradient (cont'd)

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{y \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(y|x) R(x, y)]$$

A sequence probability factorizes:

$$\log \pi_{\theta}(y|x) = \sum_t \log \pi_{\theta}(y_t|x, y_{<t}).$$

Each sampled action contributes a gradient, scaled by how good the outcome was

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) R \right].$$

Policy Gradient (cont'd)

We usually replace raw reward with something lower-variance:

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \Psi_t \right]$$

Where Ψ_t can be:

- total return G_t
- return-to-go
- advantage A_t
- TD residual
- GAE estimate

Advantage form (most important)

In modern algorithms:

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) A_t \right]$$

A_t (or Ψ_t)

Learning signal.

- Positive $\rightarrow$ action better than expected $\rightarrow$ reinforce.
- Negative $\rightarrow$ worse $\rightarrow$ suppress.
- Zero $\rightarrow$ no update.

Policy gradient updates the model by increasing the log-probability of actions that produced better-than-expected outcomes and decreasing those that produced worse outcomes.

Advantage form (most important)

In modern algorithms:

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) A_t \right]$$

A_t (or Ψ_t)

Learning signal.

- Positive $\rightarrow$ action better than expected $\rightarrow$ reinforce.
- Negative $\rightarrow$ worse $\rightarrow$ suppress.
- Zero $\rightarrow$ no update.

Policy gradient updates the model by increasing the log-probability of actions that produced better-than-expected outcomes and decreasing those that produced worse outcomes.

Suppose a response contains 5 tokens:

$$o = (o_1, o_2, o_3, o_4, o_5)$$

and receives a single response-level reward r . Its advantage is

$$A = r - b,$$

where b is the baseline. The same advantage is assigned to all tokens:

$$A_1 = A_2 = A_3 = A_4 = A_5 = A$$

The objective is

$$J(\theta) = \frac{1}{5} \sum_{t=1}^5 A \log \pi_{\theta}(o_t | o_{<t}).$$

If

$$A > 0,$$

all five tokens are reinforced:

$$P(o_1) \uparrow, \quad P(o_2 | o_1) \uparrow, \quad P(o_3 | o_{<3}) \uparrow, \quad P(o_4 | o_{<4}) \uparrow, \quad P(o_5 | o_{<5}) \uparrow.$$

If

$$A < 0,$$

all five tokens are discouraged:

$$P(o_1) \downarrow, \quad P(o_2 | o_1) \downarrow, \quad P(o_3 | o_{<3}) \downarrow, \quad P(o_4 | o_{<4}) \downarrow, \quad P(o_5 | o_{<5}) \downarrow.$$

So,

$$A > 0 \Rightarrow \text{increase probabilities of all tokens}$$

$$A < 0 \Rightarrow \text{decrease probabilities of all tokens}$$

$$o = (o_1, o_2, o_3, o_4, o_5),$$

the **supervised fine-tuning (SFT) objective** can also be written as $J(\theta)$, which we maximize:

$$J_{\text{SFT}}(\theta) = \frac{1}{5} \sum_{t=1}^5 \log \pi_{\theta}(o_t \mid o_{<t})$$

Explicitly,

$$J_{\text{SFT}}(\theta) = \frac{1}{5} [\log P(o_1) + \log P(o_2 \mid o_1) + \log P(o_3 \mid o_{<3}) + \log P(o_4 \mid o_{<4}) + \log P(o_5 \mid o_{<5})].$$

Maximizing this objective gives

$$P(o_1) \uparrow, \quad P(o_2 \mid o_1) \uparrow, \quad \dots, \quad P(o_5 \mid o_{<5}) \uparrow$$

So you can see the connection very clearly:

$$J_{\text{SFT}} = \frac{1}{5} \sum_t \log \pi_{\theta}(o_t \mid o_{<t})$$

versus

$$J_{\text{RL}} = \frac{1}{5} \sum_t A \log \pi_{\theta}(o_t \mid o_{<t})$$

SFT is essentially always saying **"make these demonstrated tokens more likely,"** whereas the RL objective can say **"make these tokens more likely"** when $A > 0$ or **"make them less likely"** when $A < 0$.

SFT

The 5 tokens come from a **human/reference response**:

$$o = (o_1, \dots, o_5) \sim \text{human data.}$$

We maximize

$$J_{\text{SFT}}(\theta) = \frac{1}{5} \sum_{t=1}^5 \log \pi_{\theta}(o_t \mid o_{<t})$$

So the model is told:

“Make the human’s tokens more likely.”

RL

The 5 tokens are **sampled from the model**:

$$o = (o_1, \dots, o_5) \sim \pi_{\theta_{\text{old}}}(\cdot \mid x)$$

Then we evaluate the generated response, get reward r , and derive advantage A .

Conceptually, the objective is

$$J_{\text{RL}}(\theta) = \frac{1}{5} \sum_{t=1}^5 A \log \pi_{\theta}(o_t \mid x, o_{<t})$$

Thus:

$A > 0 \Rightarrow$ make the model’s sampled tokens more likely

and

$A < 0 \Rightarrow$ make the model’s sampled tokens less likely.

High-level framework

1. **Sample:** pick prompts x , sample completions $y \sim \pi_\theta(\cdot|x)$.
2. **Score:** compute reward $R(x, y)$ (optionally add KL penalty).
3. **Assign credit:** produce a per-token learning signal (this is where algorithms differ).
4. **Update:** increase probability of good tokens/sequences, decrease probability of bad ones.

Core identity (intuition):

We can push probability mass toward actions that led to high reward by using

$$\nabla_\theta \log \pi_\theta(\text{chosen action})$$

as the "direction" and reward/advantage as the "scale."

Policy gradients update the model by changing the log-probabilities of what it sampled, where the change is scaled by how good the outcome was.

High-level framework (cont'd)

- **Return G_t :** "how good the outcome was from time t onward."
 - In single-final-reward RLHF, $G_t = R$ for all tokens.
- **Baseline b_t :** something that you subtract to reduce variance.
 - Baseline does not change the expected gradient, but it reduces noise.
- **Advantage A_t :** "better-than-expected"

$$A_t = G_t - b_t.$$

Key takeaway: in single-final-reward RLHF, most differences come from *how you choose b_t and how you constrain updates.*

Think of G_t as the score from your teacher
for each essay you write

Think of b_t as your own internal expectation
("I usually get an 80 on essays like this")

Return

General discounted return

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}.$$

If the episode terminates at time T , this becomes

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-t-1} R_T.$$

When $\gamma = 1$

$$G_t = R_{t+1} + R_{t+2} + \dots + R_T.$$

In RLHF with a single terminal reward

$$R_1 = 0, \dots, R_{T-1} = 0, \quad R_T = R.$$

So for every timestep t ,

$$G_t = R.$$

The return is the total future reward associated with a token/action.

For the same 5-token response,

$$o = (o_1, o_2, o_3, o_4, o_5),$$

suppose each token is an action:

$$a_t = o_t.$$

In general RL, the return at token t is

$$G_t = \sum_{k=t}^T \gamma^{k-t} r_k$$

where r_k is the reward at step k and γ is the discount factor.

For example,

$$G_1 = r_1 + \gamma r_2 + \gamma^2 r_3 + \gamma^3 r_4 + \gamma^4 r_5,$$

$$G_2 = r_2 + \gamma r_3 + \gamma^2 r_4 + \gamma^3 r_5,$$

and

$$G_5 = r_5.$$

For LLM RL

Usually, we **do not receive a reward after every token**. The model generates the entire response and then receives one reward r :

$$o_1, o_2, o_3, o_4, o_5 \xrightarrow{\text{reward model/verifier}} r.$$

So you can think of

$$r_1 = r_2 = r_3 = r_4 = 0, \quad r_5 = r.$$

With $\gamma = 1$,

$$G_1 = G_2 = G_3 = G_4 = G_5 = r.$$

Then the advantage is roughly **return minus baseline**:

$$A_t = G_t - b$$

and because every token has the same return,

$$A_1 = A_2 = A_3 = A_4 = A_5 = r - b.$$

KL penalty per token (reward shaping)

RLHF usually adds a per-token KL penalty to keep the policy close to a reference model.

So per-token reward becomes:

$$r_t = -\beta \text{KL}_t, \quad r_T = R(x, y) - \sum_t \beta \text{KL}_t.$$

KL penalty per token (reward shaping)

$$R_{\text{shaped}} = R_{\text{model}} - \beta \sum_t \text{KL}_t$$

So the KL is computed **per token**, then summed over the sequence.

Policy Gradient Algorithms (cont'd)

- REINFORCE
- RLOO (REINFORCE Leave One Out)
- PPO (Proximal Policy Optimization)

Method	G_t	b_t	Advantage shape
REINFORCE	R	batch mean	same for all tokens
RLOO	R	group mean	same for all tokens
PPO	R	$V(s_t)$	different per token

- REINFORCE $\rightarrow A_t = G_t - b$
- RLOO $\rightarrow A_t = R - \text{leave-one-out baseline}$
- PPO $\rightarrow A_t$ from value function / GAE + clipped ratio

REINFORCE

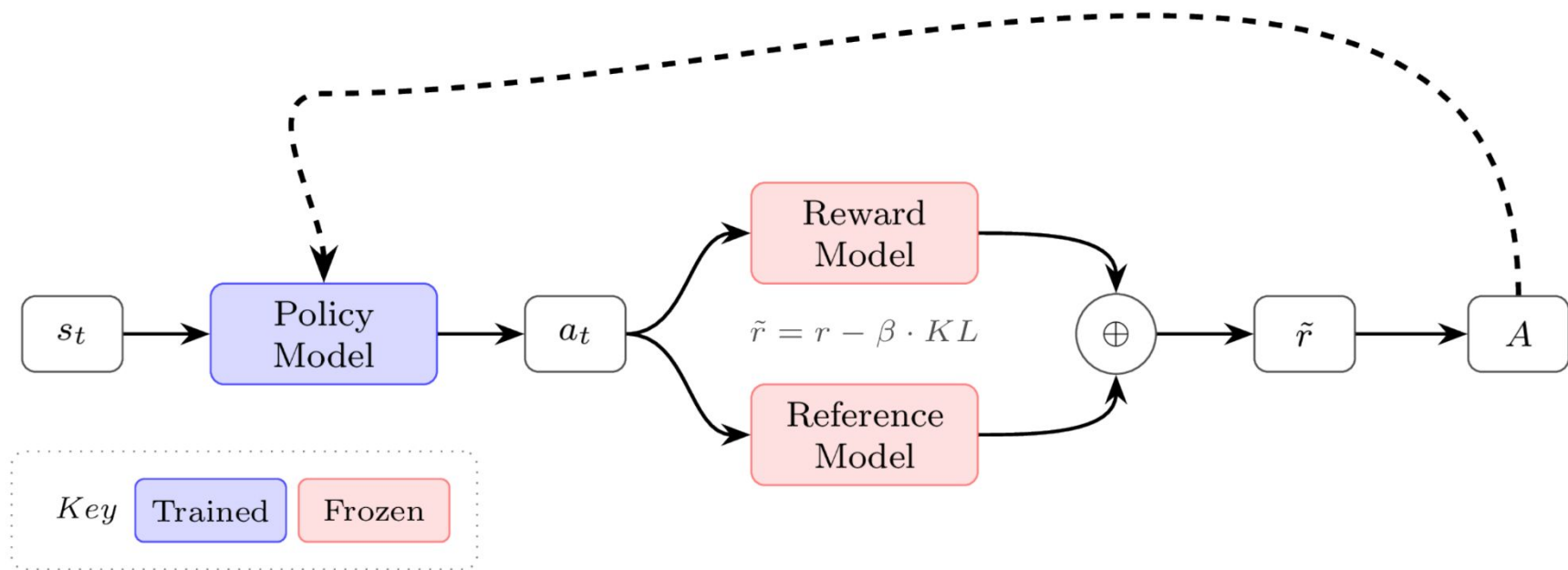


Figure 17: Basic REINFORCE architecture for language models. The shaped reward combines the reward model score with a KL penalty from the reference model. We build on this structure throughout the chapter.

REINFORCE (cont'd)

After computing the shaped reward (a single scalar), REINFORCE uses:

$$A_t = R_{\text{shaped}} - b.$$

So:

- Even though KL was per-token,
- It is **aggregated into one scalar reward**,
- Then that scalar is broadcast to all tokens.

REINFORCE (cont'd)

$b_t = \text{mean reward over batch.}$

So if your batch has rewards $[R_1, R_2, R_3]$,

$$b_t = \frac{R_1 + R_2 + R_3}{3}.$$

This baseline is the same for all tokens in that sequence.

Then:

$$G_t - b_t = R - \text{batch mean.}$$

So every token in that completion gets the same advantage.

REINFORCE (cont'd)

Intuition

- Sample a completion.
- If reward is high, reinforce every token in that completion.
- If reward is low, suppress every token in that completion.

Update shape (minimal form)

$$\sum_t \nabla_{\theta} \log \pi_{\theta}(y_t | x, y_{<t}) (R - b)$$

REINFORCE (cont'd)

It is a **Monte Carlo** gradient estimator: it uses sampled outcomes.

With a single final reward, credit is **broadcast** to all tokens.

Main weakness: **high variance**, because one scalar reward must explain many tokens.

RLOO

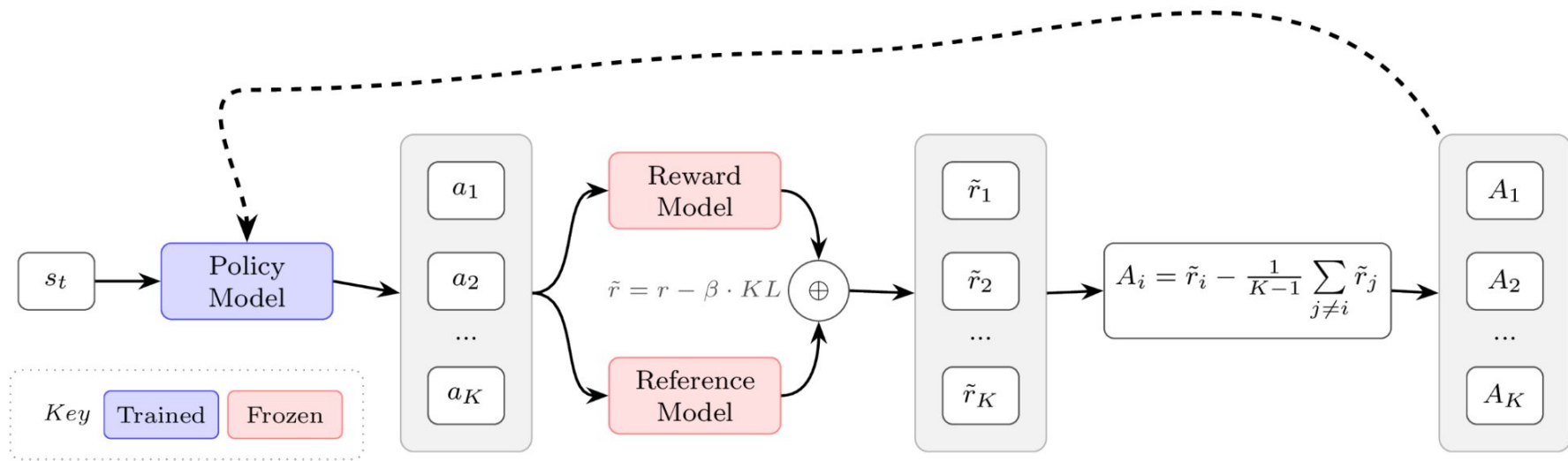


Figure 18: REINFORCE Leave-One-Out (RLOO) architecture. Multiple completions per prompt provide a leave-one-out baseline for advantage estimation without learning a value function.

RLOO (cont'd)

If you generate multiple completions per prompt:

b = mean reward of other completions.

Still one scalar per completion.

Still identical across tokens.

RLOO (cont'd)

Motivation

REINFORCE is noisy because the baseline is crude (often a batch average). RLOO improves the baseline by making it **prompt-conditional**.

Setup

- For one prompt x , sample K completions $y^{(1)}, \dots, y^{(K)}$.
- Score each completion: $R^{(k)} = R(x, y^{(k)})$.

Baseline

For completion k , the baseline is the mean reward of the *other* completions:

$$b^{(k)} = \frac{1}{K-1} \sum_{i \neq k} R^{(i)}.$$

Advantage

$$A^{(k)} = R^{(k)} - b^{(k)}.$$

RLOO (cont'd)

RLOO computes:

$$A^{(k)} = R_{\text{shaped}}^{(k)} - \text{mean of other rewards.}$$

Again:

- Shaped reward includes summed KL.
- But the advantage is one scalar per completion.
- That scalar is broadcast to all tokens.

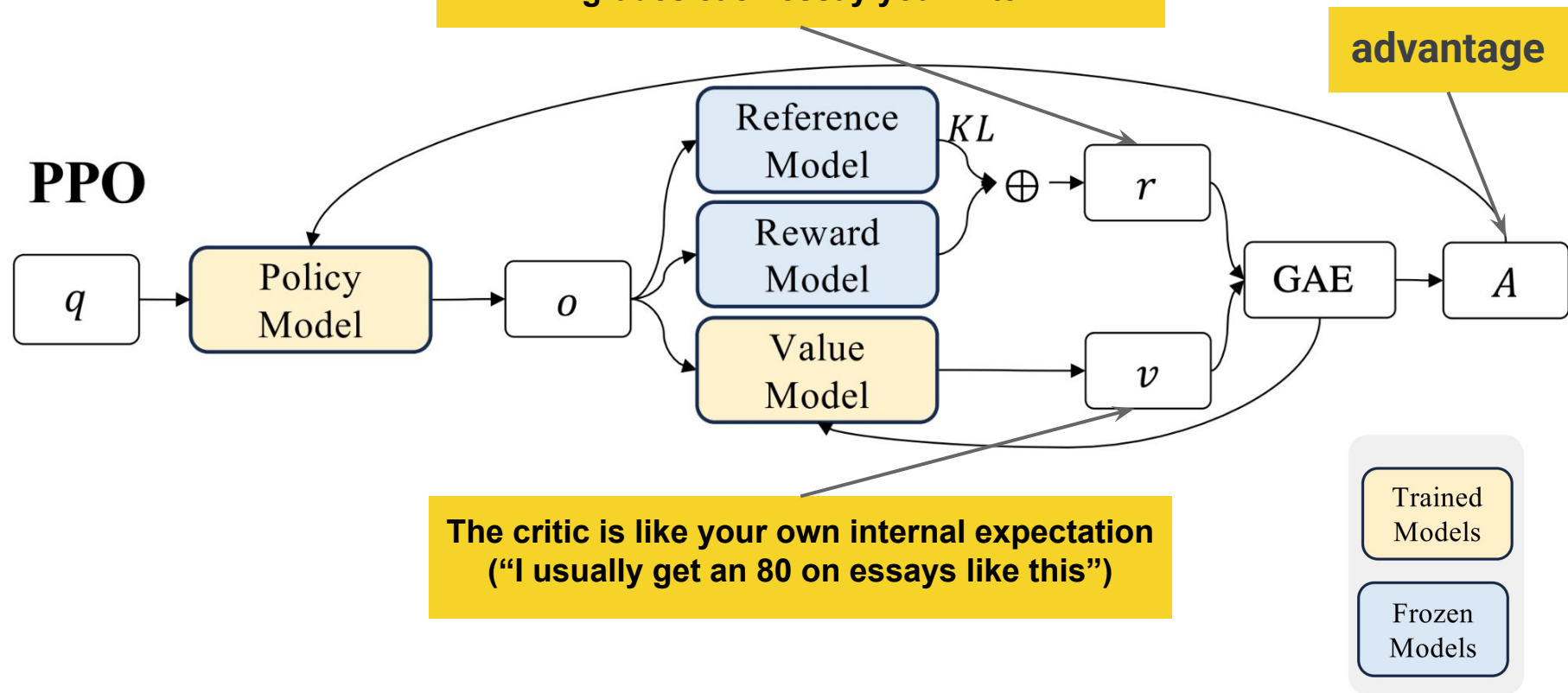
So still no token-level differentiation.

- RLOO keeps the simplicity of REINFORCE.
- It reduces variance because it compares completions **within the same prompt**, which removes prompt difficulty effects.
- It still assigns a **sequence-level** advantage that is broadcast to tokens.

Quick intuition line

RLOO turns “is this good?” into “is this better than the other tries for the same question?”

PPO



PPO (stable updates + token-level credit)

A. PPO idea #1: do not move the policy too far (the ratio + clipping)

Define the ratio:

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)} = \exp(\log \pi_\theta - \log \pi_{\text{old}}).$$

PPO optimizes a clipped objective that prevents ρ_t from leaving $[1 - \epsilon, 1 + \epsilon]$ in a way that would create overly large updates.

- π_{old} is the policy that generated the batch.
- You can do multiple gradient steps on the same batch because importance ratios correct for drift.
- Clipping creates a trust region that stabilizes training.

B. PPO idea #2: a learned value function gives token-level baselines

Value model predicts:

$$V(s_t) \approx \mathbb{E}[G_t \mid s_t].$$

Then advantage can be computed with TD or GAE (you can pick one to focus on):

- **TD residual:**

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t).$$

- **GAE:**

$$A_t^{GAE} = \delta_t + \gamma\lambda\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \dots$$

- In single-final-reward RLHF, the final reward enters at the end, and the value function propagates it backward through advantages.
- PPO has two losses: **policy loss** (clipped) and **value loss** (fit V).

PPO with Temporal-Difference (TD) error

Final advantage

$$A_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

This differs across tokens.

Meaning

Was this specific action better or worse than expected at this timestep?

- Positive $\rightarrow$ reinforce this token
- Negative $\rightarrow$ suppress this token

PPO with Generalized Advantage Estimation (GAE)

Final advantage

$$A_t = \delta_t + \gamma\lambda\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \dots$$

where

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t).$$

Meaning

A smoothed measure of how much better this action was, accounting for both immediate and future prediction errors.

- λ controls how far the reward propagates backward.
- Provides smoother and lower-variance credit assignment.

Core PPO policy objective (clipped)

Define the ratio:

$$\rho_t(\theta) = \frac{\pi_{\theta}(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)}.$$

The PPO clipped objective is:

$$L^{\text{policy}}(\theta) = \mathbb{E} \left[\min \left(\rho_t(\theta) A_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t \right) \right]$$

This is what we maximize.

Thank you!